Аналитический пайплайн IT-вакансий: DevOps

Домашнее задание №3 · объектное хранилище, MapReduce, DuckDB, профилирование нагрузки

Дисциплина	Data Engineering, ИТМО
Вариант	DevOps
Хранилище	RustFS 1.0.0, бакет hh-lake, объект raw/vacancies.csv
Данные	it_vacancies_2024.csv · 229 387 вакансий · 808.4 MiB
Стек	Python 3.14, boto3, DuckDB 2.0, psutil, matplotlib, ReportLab
Дата запусков	26.10.2026

Отчёт содержит фактические выводы команд, контрольные числа, результаты профилирования времени и памяти, а также скриншоты хранилища. Все числа получены на локальном запуске 26.10.2026.

Содержание

Содержание . . . 2
Ключевые результаты . . . 4
1. Постановка задачи и выбранный вариант . . . 4
Структура репозитория: . . . 5
2. Инфраструктура и загрузка данных . . . 5
2.1. Качество исходного CSV . . . 8
3. Методика расчёта навыков . . . 8
MapReduce v1 (наивная база). . . . 9
MapReduce v2 (чанки). . . . 9
DuckDB SQL (независимая реализация). . . . 9
4. Результаты: частоты навыков, пар и троек . . . 10
4.1. Почему пары и тройки растут быстрее навыков . . . 11
4.2. Верификация: v1 = v2 = DuckDB . . . 12
5. Профилирование нагрузки . . . 12
Интерпретация: . . . 12
6. Бонус: оптимизация через curated-витрину . . . 13
7. Влияние навыков на зарплату . . . 14
7.1. Пары, повышающие зарплату . . . 15
7.2. Пары, понижающие зарплату . . . 15
7.3. Тройки, повышающие зарплату . . . 15
7.4. Тройки, понижающие зарплату . . . 16
Что видно в данных: . . . 16
8. Региональный анализ . . . 17
8.1. Навыки и пары/тройки по регионам . . . 17
Наблюдения: . . . 18
9. Опыт и разведочный анализ . . . 18
9.1. EDA: что ещё есть в датасете . . . 20
10. Выводы . . . 21
Приложение A. Выводы команд . . . 22
A.1. Проверка окружения . . . 22
A.2. MapReduce v1 . . . 23
A.3. MapReduce v2 . . . 24
A.4. DuckDB SQL . . . 25
A.5. Верификация . . . 26

A.6. Зарплатный анализ 27
A.7. Регионы 29
A.8. Опыт 30
A.9. Профилирование (итог) 30
A.10. Бенчмарк оптимизации 30
A.11. Графики 31
Приложение B. Воспроизводимость 31
Приложение C. Ограничения и честность измерений 31

Ключевые результаты

Сводка ниже позволяет проверить отчёт по независимым контрольным точкам: число вакансий варианта, уникальные комбинации навыков, суммарные частоты, совпадение трёх реализаций, медианные зарплаты и профиль нагрузки.

Показатель	Значение
Файл в хранилище	s3://hh-lake/raw/vacancies.csv · 847 619 384 B (808.35 MiB)
Строк / валидных записей	229 389 / 229 387 (1 пустая, 1 битая 45-польная)
DevOps-вакансий (role)	5 158
Из них с навыками	3 534
Уникальные навыки / пары / тройки	1 401 / 20 454 / 144 024
Суммарные частоты k=1/2/3	28 336 / 134 651 / 506 549
Совпадение v1 = v2 = DuckDB	да, 9 из 9 сравнений
Медиана зарплаты (навыки + ЗП)	210 000 ₽ (n = 787)
Профиль v1 / v2 / DuckDB	13.31 с / 196 MiB · 12.99 с / 132 MiB · 8.52 с / 1 419 MiB
Оптимизация (curated CSV)	6.17× быстрее, 1.17× меньше пиковой памяти
Регионы	Москва 250 000 ₽ · Санкт-Петербург 210 000 ₽ · Новосибирск 172 000 ₽

Все артефакты (CSV, SQL, манифесты, логи, графики) сохранены локально в каталоге data/ и в объектном хранилище в префиксах results/, manifests/ и curated/.

1. Постановка задачи и выбранный вариант

Задание требует поднять объектное хранилище, загрузить сегмент IT-вакансий, посчитать частоты навыков, пар и троек двумя независимыми способами (собственный MapReduce по чанкам и SQL DuckDB), доказать совпадение результатов, проанализировать влияние навыков на зарплату, сравнить три региона и профилировать время и память.

Вариант — **DevOps**. Основной фильтр: `role == "DevOps-инженер"`. Поле role в датасете — это категория профессии (всего 17 значений), а не заголовок вакансии; поэтому фильтр по role даёт чистый сегмент без примеси соседних ролей. Режим фильтра настраивается переменной окружения `VARIANT_FILTER_MODE=role|role_or_name|name`.

Чувствительность к фильтру (проверено тем же кодом): при `role_or_name` (role или ключевые слова в названии) в выборку попадает 7 786 вакансий, 5 335 с навыками, 1 295 с зарплатой; уникальные k=1/2/3 — 1 918 / 31 060 / 225 224, суммарные частоты — 42 470 / 202 681 / 780 467, и все три реализации совпадают и в этом режиме. В отчёте используется строгий role-режим как основной.

Структура репозитория:

```
src/common.py               общий tolerant-ридер S3, фильтр, S3-клиенты, манифесты
src/upload.py               multipart-загрузка CSV в RustFS
src/mapreduce_v1.py         наивный MapReduce (все эмиссии в памяти)
src/mapreduce_v2.py         MapReduce по чанкам (без перекрытий и дублей)
src/duckdb_skills.py        независимый SQL-расчёт в DuckDB
src/verify.py               побитовая сверка v1 == v2 == DuckDB
src/analyze_salary.py       пары/тройки, влияющие на медианную зарплату
src/analyze_regions.py      три региона + удалённая работа
src/analyze_experience.py   зарплата и навыки по грейдам
src/eda.py                  разведочный анализ датасета
src/optimized_mapreduce.py  curated-витрина и бенчмарк-воркер
src/profile.py              3 запуска каждого метода, wall time + peak RSS
src/benchmark_optimization.py  сравнение raw vs curated
src/make_charts.py, make_screenshots.py, make_report.py
src/run_all.py, check_env.py   воспроизводимый запуск всего пайплайна
tests/test_core.py          14 unit-тестов (парсинг, чанки, эквивалентность)
compose.yaml                RustFS 9010/9011 + aws-cli
```

2. Инфраструктура и загрузка данных

Хранилище поднято через Docker Compose: RustFS 1.0.0 на порту 9010 (S3 API) и 9011 (веб-консоль), aws-cli — отдельный контейнер. Бакет hh-lake создаётся клиентом автоматически, объект загружается multipart-загрузкой по 8 MiB (102 части).

```
=== Upload complete ===
  bucket: hh-lake
  key: raw/vacancies.csv
  size_bytes: 847619384
  size_mib: 808.35
  method: boto3 multipart (102 parts × 8 MiB)

Bucket contents:
  curated/devops_skills.csv                                322653 B  (0.31 MiB)
  manifests/duckdb.json                                       921 B  (0.0 MiB)
  manifests/eda.json                                         1273 B  (0.0 MiB)
  manifests/experience.json                                  1750 B  (0.0 MiB)
  manifests/mapreduce_v1.json                                 851 B  (0.0 MiB)
  manifests/mapreduce_v2.json                                 902 B  (0.0 MiB)
  manifests/optimized_prepare.json                           1133 B  (0.0 MiB)
  manifests/optimized_worker.json                             715 B  (0.0 MiB)
  manifests/regions.json                                     2793 B  (0.0 MiB)
  manifests/salary.json                                      1380 B  (0.0 MiB)
  raw/vacancies.csv                                      847619384 B  (808.35 MiB)
  results/duckdb/query.sql                                   1970 B  (0.0 MiB)
  results/duckdb/skills_1.csv                               29671 B  (0.03 MiB)
  results/duckdb/skills_2.csv                              625816 B  (0.6 MiB)
  results/duckdb/skills_3.csv                             5648732 B  (5.39 MiB)
  results/eda/salary_histogram.csv                            142 B  (0.0 MiB)
  results/eda/skills_count_distribution.csv                   202 B  (0.0 MiB)
  results/eda/summary.csv                                     533 B  (0.0 MiB)
  results/eda/top_areas.csv                                   367 B  (0.0 MiB)
  results/eda/top_devops_areas.csv                            332 B  (0.0 MiB)
  results/eda/top_devops_skills.csv                           408 B  (0.0 MiB)
  results/eda/top_employers.csv                               164 B  (0.0 MiB)
  results/eda/top_employers_all.csv                           178 B  (0.0 MiB)
  results/eda/top_experiences.csv                             124 B  (0.0 MiB)
  results/eda/top_roles.csv                                   826 B  (0.0 MiB)
  results/eda/top_schedules.csv                               170 B  (0.0 MiB)
  results/eda/top_sources.csv                                  23 B  (0.0 MiB)
  results/eda/top_spheres.csv                                  13 B  (0.0 MiB)
  results/eda/top_status.csv                                   40 B  (0.0 MiB)
  results/experience/summary.csv                              304 B  (0.0 MiB)
  results/experience/top_skills_per_experience.csv           1379 B  (0.0 MiB)
  results/mapreduce_v1/skills_1.csv                         29671 B  (0.03 MiB)
  results/mapreduce_v1/skills_2.csv                        625816 B  (0.6 MiB)
  results/mapreduce_v1/skills_3.csv                       5648732 B  (5.39 MiB)
  results/mapreduce_v2/skills_1.csv                         29671 B  (0.03 MiB)
  results/mapreduce_v2/skills_2.csv                        625816 B  (0.6 MiB)
  results/mapreduce_v2/skills_3.csv                       5648732 B  (5.39 MiB)
  results/optimized_mapreduce/skills_1.csv                  29671 B  (0.03 MiB)

  results/optimized_mapreduce/skills_2.csv                 625816 B  (0.6 MiB)
  results/optimized_mapreduce/skills_3.csv                5648732 B  (5.39 MiB)
  results/regions/overview.csv                                560 B  (0.0 MiB)
  results/regions/remote_overview.csv                         315 B  (0.0 MiB)
  results/regions/Москва_demand.csv                           146 B  (0.0 MiB)
  results/regions/Москва_pairs.csv                            139 B  (0.0 MiB)
  results/regions/Москва_skills.csv                           102 B  (0.0 MiB)
  results/regions/Москва_triples.csv                          178 B  (0.0 MiB)
  results/regions/Новосибирск_demand.csv                       62 B  (0.0 MiB)
  results/regions/Новосибирск_pairs.csv
... (обрезано, полный лог в report/logs/)
```

Контроль целостности: размер объекта после загрузки равен локальному файлу — 847 619 384 байта; SHA-256 локального файла `fb6a23f662c0b458e7d45d723862df5406cc25366164f4bb8246aad1cd4e83ef`. Multipart ETag `efb654589959327470 01aa004592d19c-102` содержит суффикс -102, то есть ровно 102 части, что совпадает с числом частей загрузки.

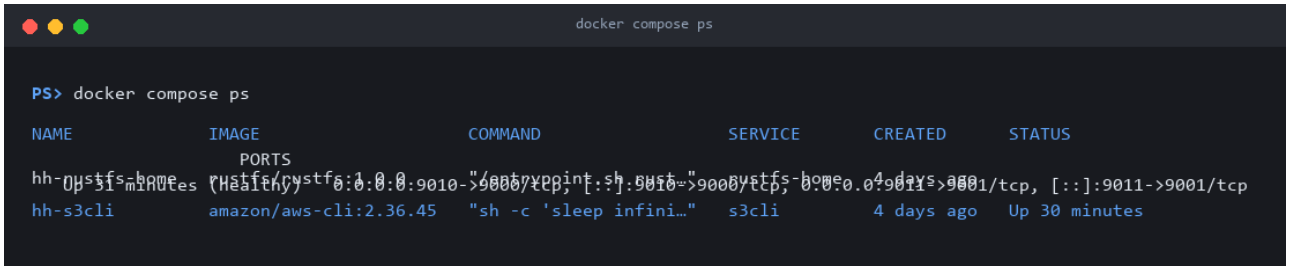


Рис. 1. docker compose ps: контейнер RustFS в состоянии healthy.

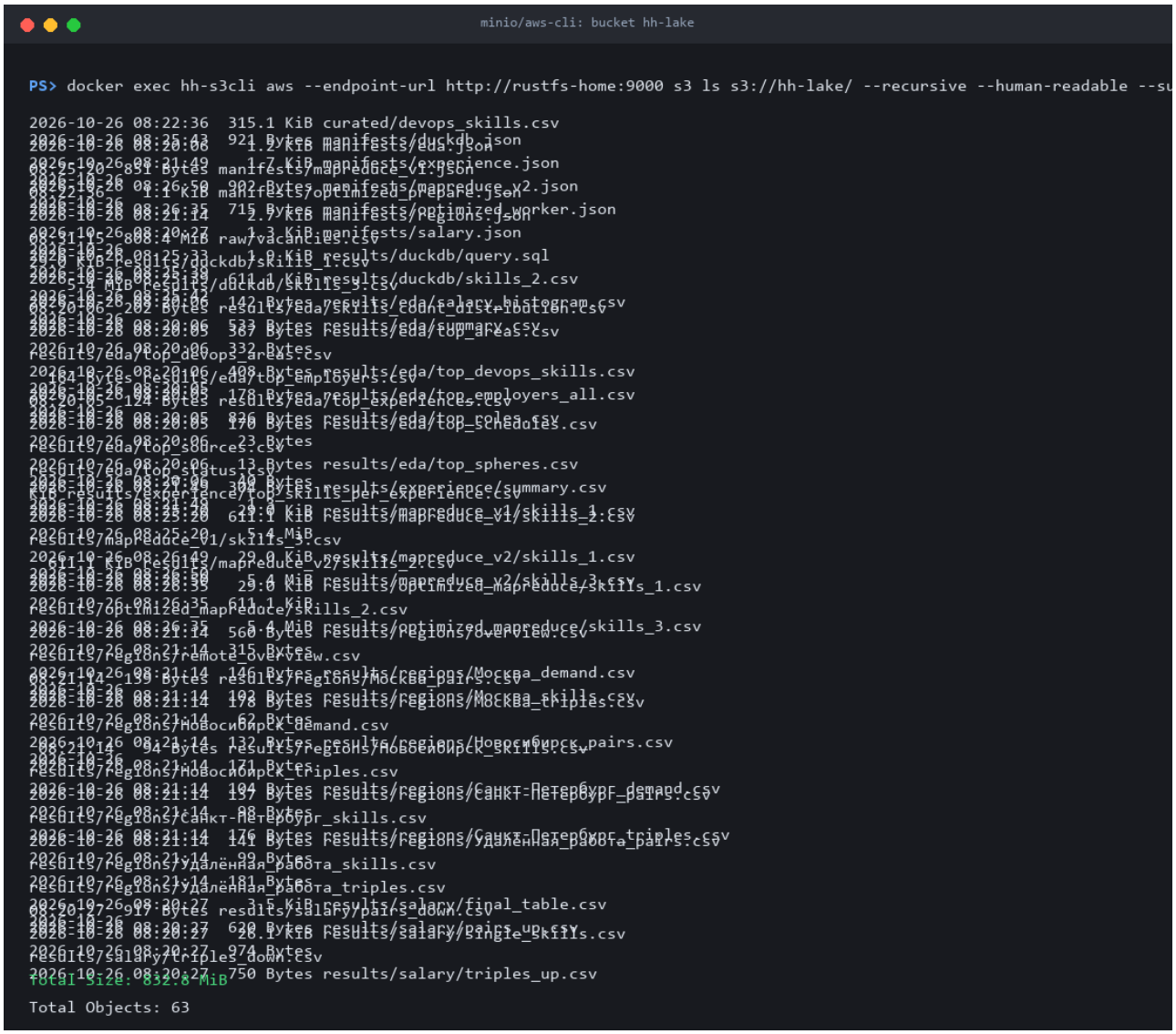


Рис. 2. Содержимое бакета hh-lake: raw/vacancies.csv 808.4 MiB, curated-витрина, results/ и manifests/ (aws-cli, реальный вывод).

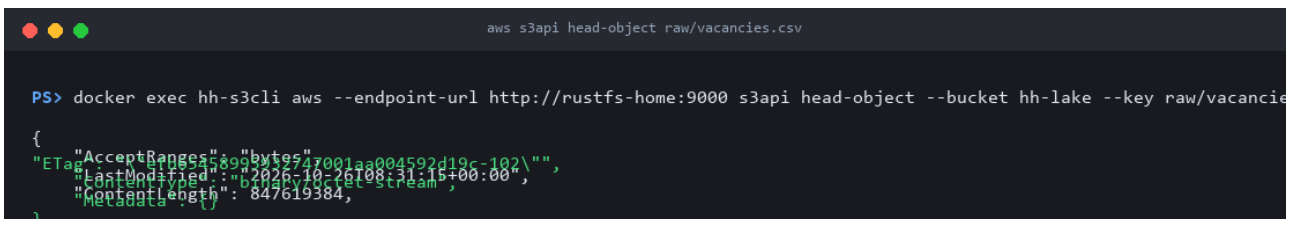


Рис. 3. head-object: размер и ETag объекта raw/vacancies.csv.

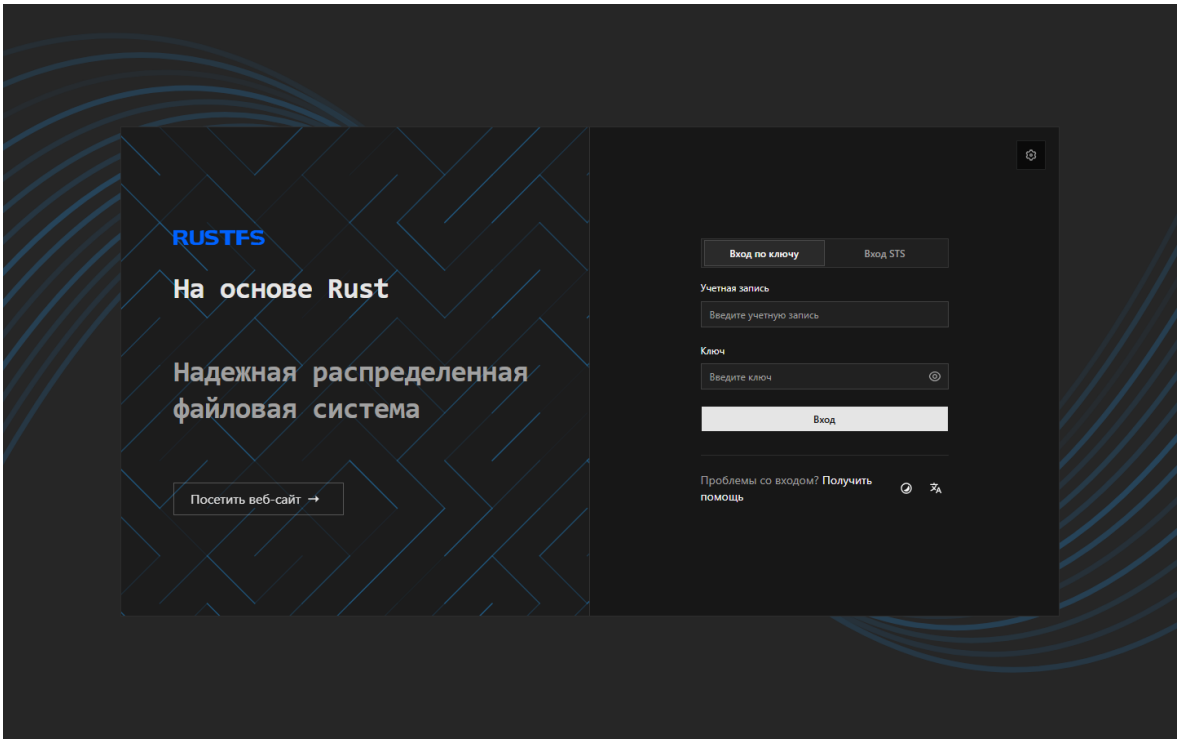


Рис. 4. Веб-консоль RustFS на http://localhost:9011 (headless-снимок реальной страницы).

2.1. Качество исходного CSV

Файл имеет нестандартные особенности, из-за которых «наивный» csv.DictReader даёт пустой результат: первая физическая строка пустая, заголовка нет, а одна запись содержит 45 полей вместо 23 (vacancy_id 106357315). Поэтому во всех реализациях используется явная схема из 23 полей и tolerant-разбор: пустые строки, заголовок и строки с неверным числом полей пропускаются и считаются отдельно.

Метрика	Значение
Физических строк	229 389
Валидных записей	229 387
Пропущено пустых	1
Пропущено битых	1 (45 полей вместо 23)
Уникальных vacancy_id	229 387
Колонок	23

Схема: source, vacancy_id, status, snapshot_date, published_at, area, employer_id, employer_inn, employer_ogrn, employer_name, name, role, sphere, salary_from, salary_to, currency, gross, experience, schedule, employment, key_skills, description, url. Поле employer_name в датасете пустое, поэтому работодатели в EDA считаются по employer_id.

3. Методика расчёта навыков

Навыки хранятся в key_skills через точку с запятой. Нормализация: trim, нижний регистр, Unicode NFC, удаление пустых токенов и дублей внутри вакансии. Для каждой вакансии с непустым набором навыков генерируются все сочетания размера k = 1, 2, 3; комбинации внутри вакансии сортируются, поэтому пары и тройки не зависят от порядка навыков в исходном поле.

MapReduce v1 (наивная база).

Один потоковый проход по объекту; все эмиссии (combo, 1) складываются в список в памяти, затем reducer считает частоты. Это сознательно неэкономная реализация: она показывает цену материализации всех эмиссий. По сравнению с исходным вариантом строки не держатся в памяти целиком, но эмиссии — держатся. Всего 669 536 эмиссий.

MapReduce v2 (чанки).

Объект читается ranged-GET-запросами по 8 MiB. Границы чанков выравниваются по переводу строки: последняя неполная строка переносится в следующий чанк, а строка длиннее чанка собирается целиком. Это исключает и потерю, и двойной учёт строк. В каждом чанке сначала работает дешёвый байтовый префильтр по подстрокам фильтра, затем точный разбор и подсчёт Counter; частичные счётчики объединяются.

DuckDB SQL (независимая реализация).

SQL читает тот же объект напрямую из S3 через httpfs. Условие варианта написано вручную; CSV читается с header=false и явным списком имён колонок, ignore_errors=true. Навыки разворачиваются через UNNEST и string_split, пары и тройки строятся самосоединением с условиями s1 < s2 < s3, что эквивалентно сочетаниям без повторений.

```
WITH vacancies AS (
    SELECT row_number() OVER () AS rid, vacancy_id, role, name, key_skills
    FROM read_csv('s3://hh-lake/raw/vacancies.csv', header=false, names=['source', 'vacancy_id', 'status', 'snap
shot_date', 'published_at', 'area', 'employer_id', 'employer_inn', 'employer_ogrn', 'employer_name', 'name', 'ro
le', 'sphere', 'salary_from', 'salary_to', 'currency', 'gross', 'experience', 'schedule', 'employment', 'key_ski
lls', 'description', 'url'], all_varchar=true, max_line_size=20000000, ignore_errors=true)
    WHERE TRIM(COALESCE(role, '')) = 'DevOps-инженер'
),
stacks AS (
    SELECT rid, list_sort(list_distinct(list(lower(trim(skill))))) AS skills
    FROM vacancies, UNNEST(string_split(COALESCE(key_skills, ''), ';')) AS t(skill)
    WHERE trim(skill) <> ''
    GROUP BY rid
),
single AS (
    SELECT t.s AS s1, COUNT(*) AS freq
    FROM stacks, UNNEST(skills) AS t(s)
    GROUP BY 1
),
pairs AS (
    SELECT ta.s AS s1, tb.s AS s2, COUNT(*) AS freq
    FROM stacks, UNNEST(skills) AS ta(s), UNNEST(skills) AS tb(s)
    WHERE ta.s < tb.s
    GROUP BY 1, 2
),
triples AS (
    SELECT ta.s AS s1, tb.s AS s2, tc.s AS s3, COUNT(*) AS freq
    FROM stacks, UNNEST(skills) AS ta(s), UNNEST(skills) AS tb(s), UNNEST(skills) AS tc(s)
    WHERE ta.s < tb.s AND tb.s < tc.s
    GROUP BY 1, 2, 3
)
SELECT CAST(0 AS BIGINT) AS k, 'n_matched' AS combo1, CAST(COUNT(*) AS VARCHAR) AS combo2,
       '' AS combo3, CAST(0 AS BIGINT) AS freq
FROM vacancies
UNION ALL
SELECT 0, 'n_with_skills', CAST(COUNT(*) AS VARCHAR), '', 0 FROM stacks
UNION ALL
SELECT CAST(1 AS BIGINT), CAST(s1 AS VARCHAR), CAST('' AS VARCHAR), CAST('' AS VARCHAR), CAST(freq AS BIGINT) FR
OM single
UNION ALL
SELECT CAST(2 AS BIGINT), CAST(s1 AS VARCHAR), CAST(s2 AS VARCHAR), CAST('' AS VARCHAR), CAST(freq AS BIGINT) FR
OM pairs
UNION ALL
SELECT CAST(3 AS BIGINT), CAST(s1 AS VARCHAR), CAST(s2 AS VARCHAR), CAST(s3 AS VARCHAR), CAST(freq AS BIGINT) FR
OM triples
ORDER BY k, freq DESC
```

Полный SQL сохранён в data/results_duckdb/query.sql и в s3://hh-lake/results/duckdb/query.sql.

4. Результаты: частоты навыков, пар и троек

Размер стека	Уникальных	Суммарная частота	Топ-5 комбинаций (частота)
k = 1	1 401	28 336	linux (2 190); kubernetes (1 862); docker (1 850); devops (1 319); ansible (1 252)
k = 2	20 454	134 651	docker;kubernetes (1 251); docker;linux (1 208); kubernetes;linux (1 162); ansible;linux (871); devops;linux (842)
k = 3	144 024	506 549	docker;kubernetes;linux (804); devops;docker;kubernetes (585); ansible;docker;kubernetes (552); ansible;docker;linux (546); devops;docker;linux (542)

Суммарные частоты совпадают с числом эмиссий v1: 28 336 + 134 651 + 506 549 = 669 536 — это независимая проверка того, что ни одна вакансия не потеряна и не учтена дважды.

4.1. Почему пары и тройки растут быстрее навыков

k	Уникальных	Всего вхождений	Вхождений на вакансию	Уникальные / вхождения	Рост к предыдущему k
k = 1	1 401	28 336	8.02	4.9 %	—
k = 2	20 454	134 651	38.10	15.2 %	14.6× / 4.75×
k = 3	144 024	506 549	143.34	28.4 %	7.0× / 3.76×

Рост объясняется комбинаторикой внутри вакансии. Вакансия с n навыками даёт C(n,1) = n одиночных навыков, C(n,2) = n(n−1)/2 пар и C(n,3) = n(n−1)(n−2)/6 троек. В выборке в среднем 8.02 навыка на вакансию с навыками (28 336 / 3 534), то есть уже «средняя» вакансия даёт 8 + 28 + 56 = 92 вхождения. Распределение числа навыков имеет тяжёлый хвост: одна вакансия с 25 навыками добавляет 2 300 троек. Поэтому суммарные вхождения троек (506 549) в 3.76 раза больше пар (134 651) и в 17.9 раз больше одиночных навыков (28 336), хотя уникальных троек лишь в 7.0 раз больше уникальных пар: частые комбинации (docker/kubernetes/linux) повторяются у многих вакансий и «схлопываются» в одну строку результата.

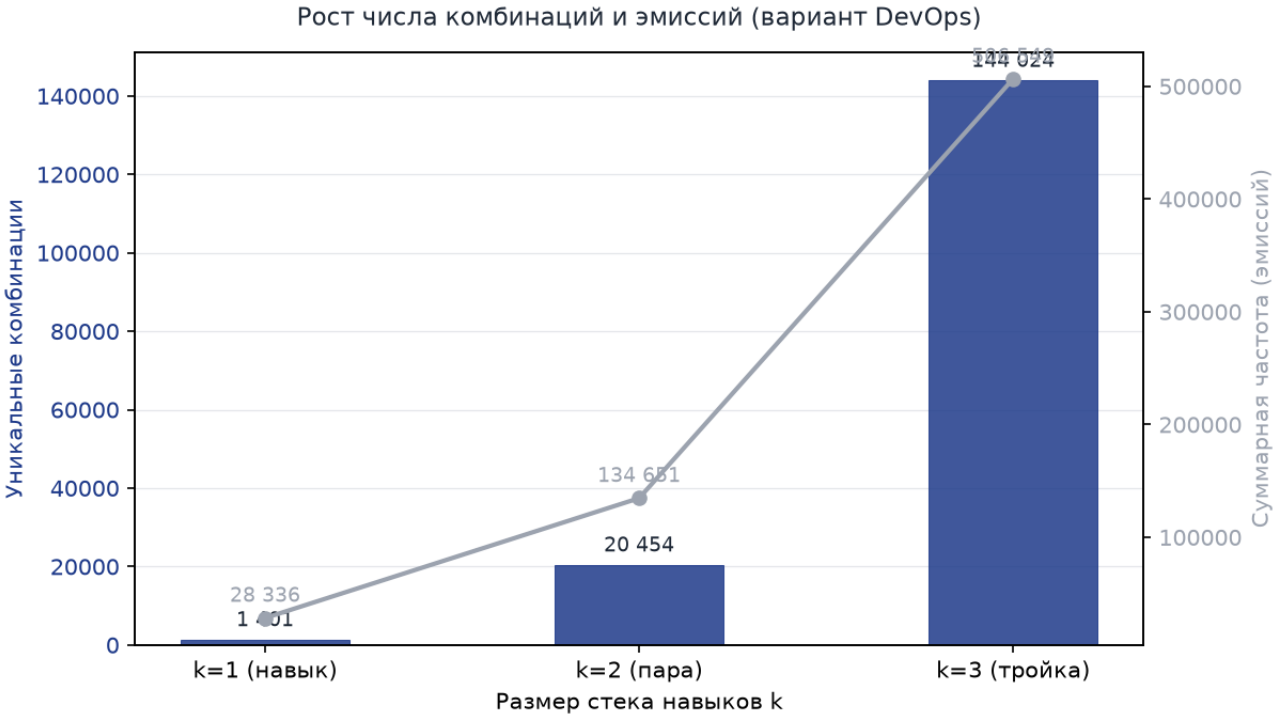


Рис. 5. Рост числа уникальных комбинаций и суммарных эмиссий при k = 1, 2, 3. Эмиссии растут почти линейно по k, уникальные комбинации — медленнее из-за повторений.

4.2. Верификация: v1 = v2 = DuckDB

k	Метод A	Метод B	Строк	Сумма частот	Результат
k = 1	v1	v2	1 401	28 336	совпало
k = 1	v1	duckdb	1 401	28 336	совпало
k = 1	v2	duckdb	1 401	28 336	совпало
k = 2	v1	v2	20 454	134 651	совпало
k = 2	v1	duckdb	20 454	134 651	совпало
k = 2	v2	duckdb	20 454	134 651	совпало
k = 3	v1	v2	144 024	506 549	совпало
k = 3	v1	duckdb	144 024	506 549	совпало
k = 3	v2	duckdb	144 024	506 549	совпало

Дополнительно verify.py сверяет файлы результатов с манифестами: число строк skills_k.csv равно n_skills_k из манифеста, режим фильтра одинаков у всех методов, rows_matched = 5 158 у v1, v2 и DuckDB. Пустые наборы результатов трактуются как ошибка (иначе «совпадение» двух пустых файлов было бы ложным успехом).



Рис. 6. Фактический вывод verify.py: 9 из 9 сравнений совпали.

5. Профилирование нагрузки

Каждый метод запускался отдельным процессом: один прогревочный запуск (исключён из статистики) и три измерительных, методы чередуются, чтобы сгладить влияние прогрева ОС и кэша RustFS. Wall time измеряется monotonic-таймером вокруг процесса; пиковая память — суммарный RSS дерева процессов (psutil) с опросом каждые 20 мс. Логи каждого запуска лежат в report/logs/profile_*.log.

Метод	Wall, с	Wall min-max	Peak RSS, MiB	Peak min-max	Успешно
MapReduce v1 (naive)	13.31	12.70–13.37	196	196–204	3/3
MapReduce v2 (optimized)	12.98	12.55–13.69	132	127–133	3/3
DuckDB SQL	8.52	7.55–9.32	1418	1418–1723	3/3

Интерпретация:

- DuckDB быстрее всех по времени (8.52 с против 12.99 с у v2, ускорение 1.53×): векторизованный движок и параллелизм, но он читает весь CSV и держит в памяти агрегаты — пик 1 419 MiB, в 10.7 раза больше, чем у потокового v2.

- MapReduce v2 почти не быстрее v1 по времени (12.99 с против 13.31 с, +2.5 %): узкое место — чтение 808 MiB из S3 и разбор 229 тыс. строк, а не reducer. Зато v2 в 1.48 раза экономнее по памяти (132 MiB против 196 MiB), потому что агрегирует частичные Counter по чанкам и не хранит 669 536 эмиссий.
- Пик v1 (196 MiB) стабилен (195.9–204.0 MiB), v2 — 127–133 MiB: оба варианта комфортно работают на машине с 15.3 GiB RAM. DuckDB в третьем запуске достиг 1 724 MiB — разброс связан с кэшем httpfs и планировщиком.
- Машина: 16 CPU, 15.34 GiB RAM, Python 3.14.5, Windows (win32), RustFS локально на loopback. Внешняя сеть не использовалась, поэтому времена отражают локальный диск и HTTP-стек, а не пропускную способность облака.

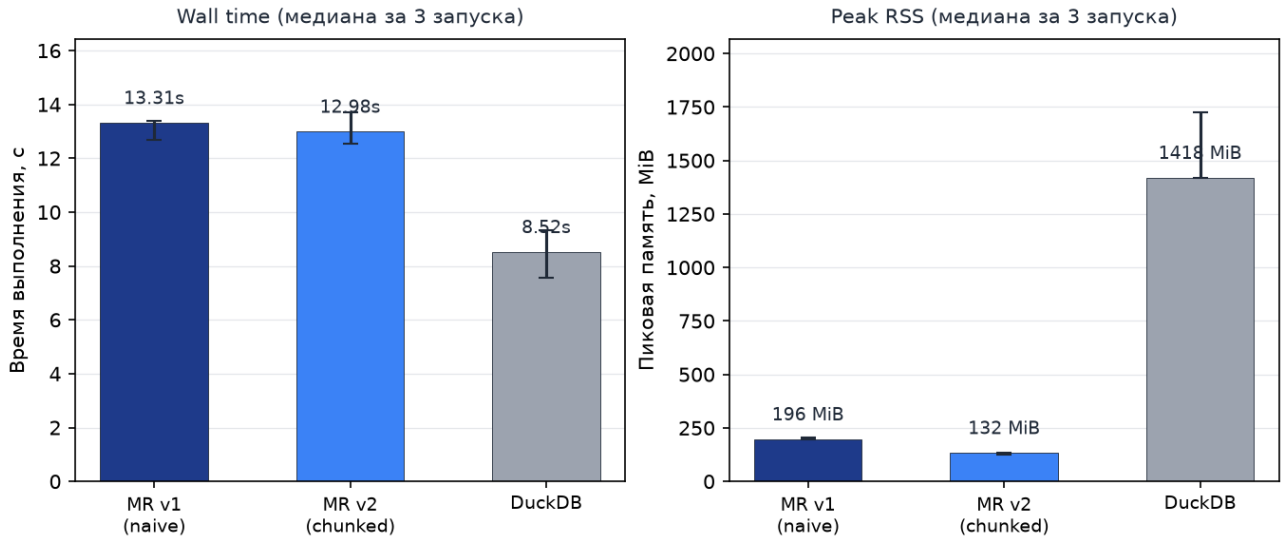


Рис. 7. Wall time и пиковая память: медианы трёх запусков, усы — min–max.

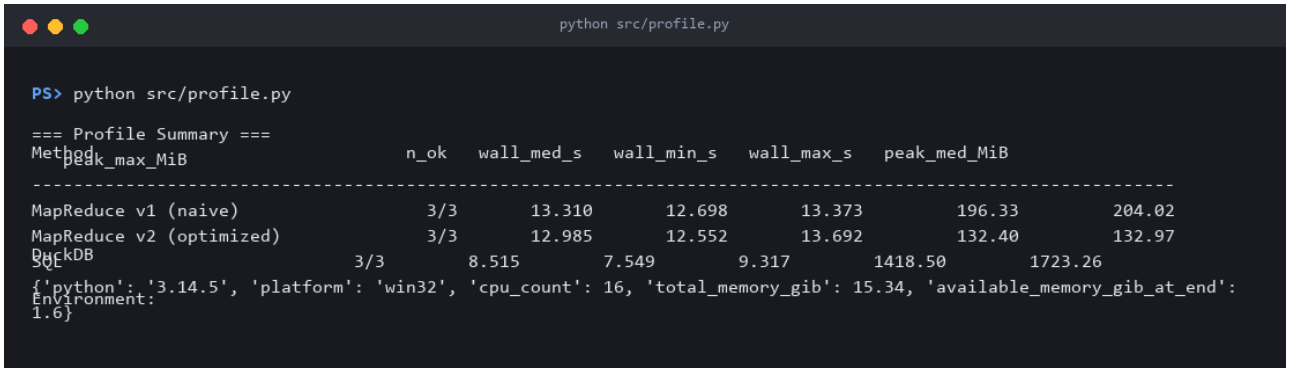


Рис. 8. Вывод профилировщика (фрагмент с итоговой таблицей).

6. Бонус: оптимизация через curated-витрину

Идея: один раз отфильтровать 808 MiB raw-файла в компактную витрину из двух колонок (vacancy_id, нормализованные навыки) и дальше считать MapReduce только по ней. Витрина содержит 3 534 строки и занимает 322 653 B (315 KiB) — в 2 627 раз меньше исходного объекта. Корректность витрины проверяется построчно: optimized_mapreduce.py verify сравнивает vacancy_id и отсортированные наборы навыков с полным проходом по raw-файлу.

Метод	Wall, с (медиана)	Peak RSS, MiB	Успешно
MapReduce v2 (raw CSV)	13.16	132	3/3

Метод	Wall, с (медиана)	Peak RSS, MiB	Успешно
MapReduce v2 (curated CSV)	2.13	113	3/3

Итог бенчмарка: **6.17× быстрее** (13.16 → 2.13 с) и **1.17× меньше пиковой памяти** (132 → 113 MiB). Прирост памяти невелик, потому что базовый Python-процесс и импорт библиотек дают постоянный вклад; главный выигрыш — в объёме читаемых данных и в отсутствии повторного разбора 23 колонок. Результаты raw и curated совпали побитово (число строк и суммы частот по k = 1, 2, 3), поэтому ускорение не куплено ценой изменения ответа.


```
{
  "action": "verify",
  "filter_mode": "role",
  "full_devops_rows": 3534,
  "curated_rows": 3534,
  "matching_rows": 3534,
  "mismatches": 0,
  "missing_in_curated": 0,
  "ok": true
}
```

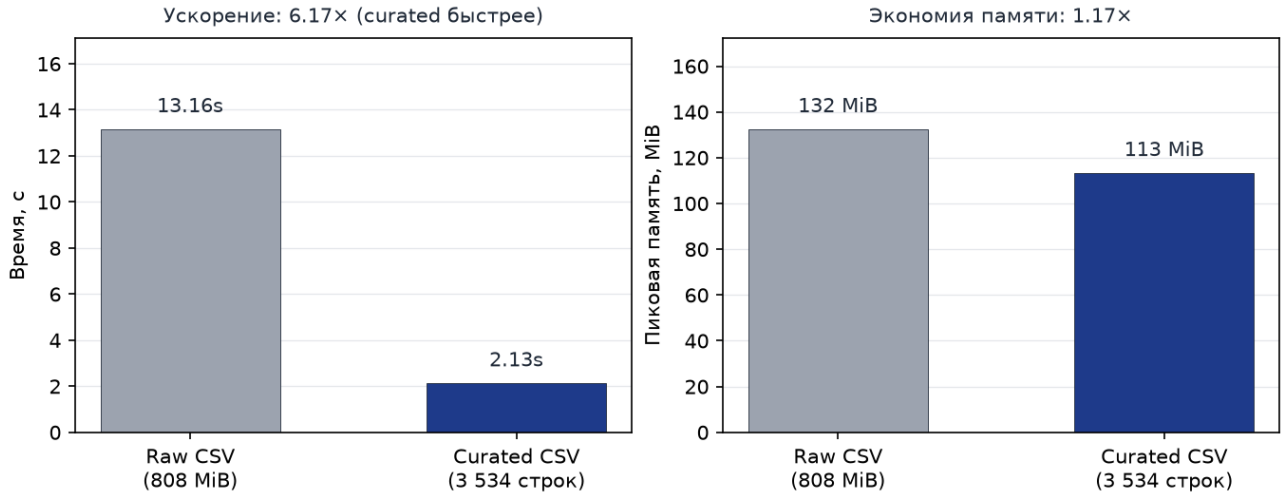



Рис. 9. Сравнение raw и curated: время и пиковая память (медианы трёх запусков).

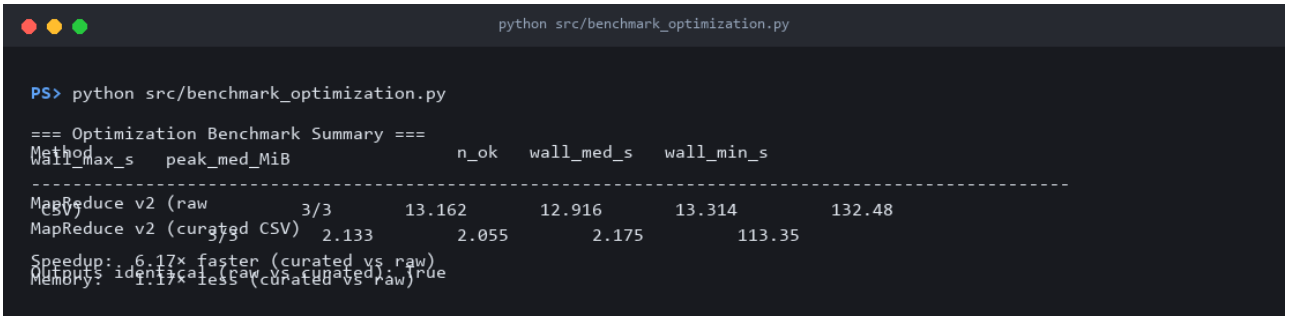


Рис. 10. Итоговая таблица бенчмарка и проверка совпадения выходов.

7. Влияние навыков на зарплату

Зарплата нормализуется в месячную сумму в рублях: берётся середина диапазона salary_from/salary_to (если задана одна граница — она), пересчёт USD 90 ₽, EUR 100 ₽, KZT 0.19 ₽; gross-суммы уменьшаются на 13 % НДФЛ. Записи без конвертируемой зарплаты (включая 1 строку в BYR) исключаются. Из 5 158 DevOps-вакансий зарплата есть у 1 110, а одновременно навыки и зарплата — у 787; медиана по этой выборке 210 000 ₽ (по всем 1 110 — 215 000 ₽).

Для пары базой считается более частый из двух навыков, для тройки — более частая из трёх пар, входящих в неё. Это требование задания: сравнивать с базовым, а не с менее частым навыком. Порог: пара — не менее 10 вакансий, тройка — не менее 8, база — не менее 15.

Показатель	Значение
DevOps-вакансий с зарплатой	1 110
С навыками и зарплатой	787
Медиана (навыки + зарплата)	210 000 ₽
Медиана (все с зарплатой)	215 000 ₽
Пар после порога	582
Троек после порога	2 880
Валюты	RUR 1 037 · USD 53 · EUR 20 · BYR 1 (не конвертируется)

7.1. Пары, повышающие зарплату

Комбинация	n	Медиана	База	Медиана базы	n базы	Δ, ₽	Δ, %
devops;sre	12	350 175 ₽	devops	216 250 ₽	348	+133 925 ₽	+61.9 %
grafana;openshift	12	350 000 ₽	grafana	237 075 ₽	224	+112 925 ₽	+47.6 %
bash;мониторинг	10	327 250 ₽	bash	215 000 ₽	219	+112 250 ₽	+52.2 %
docker;openshift	12	322 500 ₽	docker	217 500 ₽	462	+105 000 ₽	+48.3 %
linux;openshift	13	304 500 ₽	linux	200 000 ₽	478	+104 500 ₽	+52.2 %

7.2. Пары, понижающие зарплату

Комбинация	n	Медиана	База	Медиана базы	n базы	Δ, ₽	Δ, %
ansible;администрирование	11	112 500 ₽	ansible	250 000 ₽	267	-137 500 ₽	-55.0 %
atlassian jira;prometheus	10	125 000 ₽	prometheus	250 000 ₽	180	-125 000 ₽	-50.0 %
ansible;debian	11	140 000 ₽	ansible	250 000 ₽	267	-110 000 ₽	-44.0 %
debian;postgresql	11	130 500 ₽	postgresql	239 250 ₽	245	-108 750 ₽	-45.5 %
debian;linux	19	100 000 ₽	linux	200 000 ₽	478	-100 000 ₽	-50.0 %

7.3. Тройки, повышающие зарплату

Комбинация	n	Медиана	База	Медиана базы	n базы	Δ, ₽	Δ, %
git;gitlab ci;python	9	360 000 ₽	git;python	180 000 ₽	76	+180 000 ₽	+100.0 %
git;gitlab ci;postgresql	11	360 000 ₽	git;postgresql	200 000 ₽	84	+160 000 ₽	+80.0 %
gitlab ci;linux;python	9	360 000 ₽	linux;python	208 800 ₽	151	+151 200 ₽	+72.4 %
grafana;helm;terraform	8	400 625 ₽	grafana;terraform	250 000 ₽	72	+150 625 ₽	+60.2 %
ci/cd;python;zabbix	17	350 000 ₽	ci/cd;python	200 000 ₽	69	+150 000 ₽	+75.0 %

7.4. Тройки, понижающие зарплату

Комбинация	n	Медиана	База	Медиана базы	n базы	Δ, ₽	Δ, %
docker-compose;linux;prometheus	9	100 000 ₽	linux;prometheus	245 000 ₽	132	-145 000 ₽	-59.2 %
jenkins;postgresql;sql	9	100 000 ₽	jenkins;postgresql	232 125 ₽	38	-132 125 ₽	-56.9 %
elk;prometheus;администрирование серверов linux	19	125 000 ₽	elk;prometheus	250 000 ₽	64	-125 000 ₽	-50.0 %
ansible;atlassian jira;linux	10	125 000 ₽	ansible;linux	250 000 ₽	190	-125 000 ₽	-50.0 %
ansible;atlassian jira;prometheus	10	125 000 ₽	ansible;prometheus	250 000 ₽	114	-125 000 ₽	-50.0 %

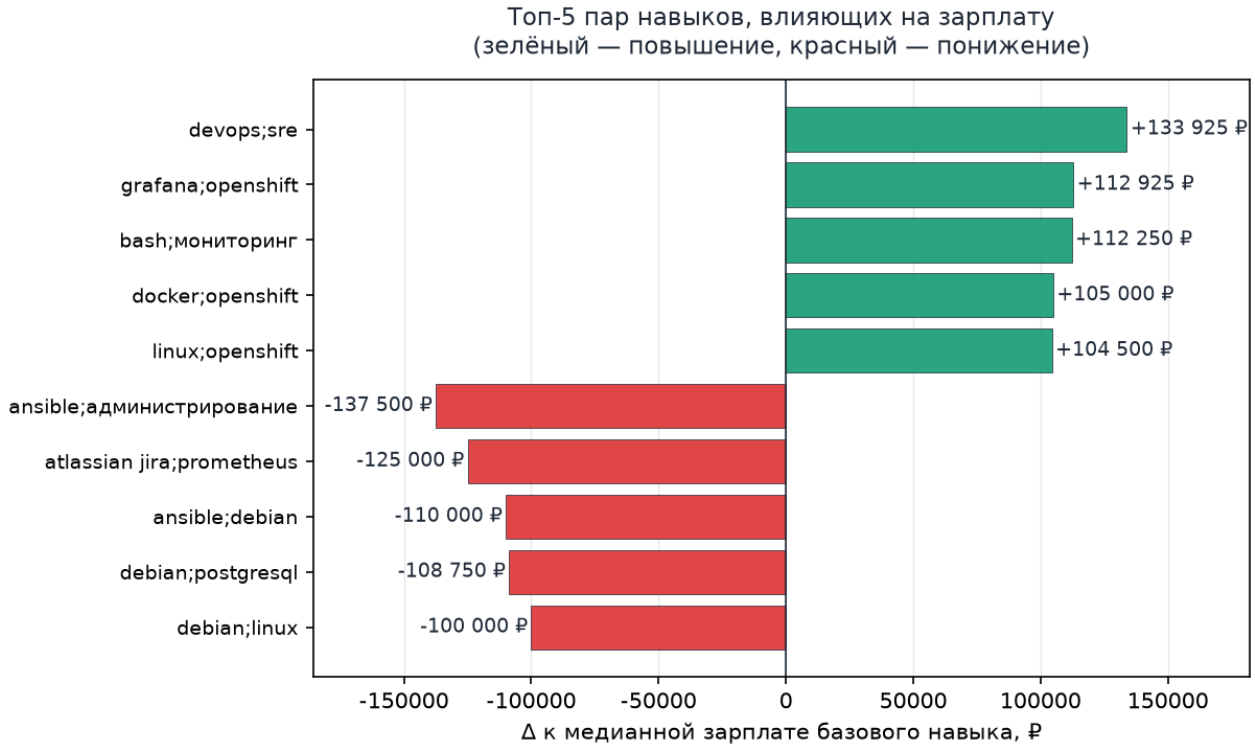


Рис. 11. Топ-5 пар, повышающих и понижающих медианную зарплату относительно базового навыка.

Что видно в данных:

- Наибольшая прибавка у пары devops + sre: медиана 350 175 ₽ при базе devops 216 250 ₽ (+61.9 %, n = 12). Близко к ней идут grafana + openshift (+47.6 %), bash + мониторинг (+52.2 %), docker + openshift (+48.3 %), git + gitlab ci (+53.3 %): это признаки зрелой платформенной/SRE-роли с CI/CD и наблюдаемостью.
- Тройки с gitlab ci, helm, terraform, zabbix и python дают +60…+100 % к базовой паре (например, git + gitlab ci + python: 360 000 ₽ против 180 000 ₽ у git + python, n = 9). Это уже не «отдельный навык», а связка «CI/CD + IaC + автоматизация», которая в выборке соответствует senior-позициям.
- Понижают зарплату ansible + администрирование (−55.0 %), ansible + debian (−44.0 %), debian + linux (−50.0 %), docker-compose + grafana (−40.9 %), atlassian jira + prometheus (−50.0 %). Здесь важна интерпретация: сами по себе Ansible или Debian — востребованные навыки, но в паре с тегами «администрирование»/«debian» они маркируют legacy-эксплуатацию и сопровождение, а не разработку платформы; такие

вакансии платят меньше.

- Ограничение: n у топовых комбинаций 8–38 вакансий, поэтому проценты — индикативные. Устойчивее вывод о направлении (IaC/CI-CD/SRE вверх, legacy-администрирование вниз), чем о конкретной величине прибавки.

8. Региональный анализ

Выбраны три региона с наибольшим числом DevOps-вакансий после Москвы и Санкт-Петербурга: Москва, Санкт-Петербург и Новосибирск. Удалённая работа показана отдельным срезом, потому что это не город; нормализация ё→е позволяет не потерять вариант «Удаленная работа».

Регион	Вакансий	С навыками	С ЗП	Доля с ЗП	Медиана	P25	P75	Ср. навыков
Москва	3 007	2 015	573	19.1 %	250 000 ₽	195 750 ₽	340 000 ₽	7.65
Санкт-Петербург	686	488	161	23.5 %	210 000 ₽	153 000 ₽	275 000 ₽	8.34
Новосибирск	160	123	48	30.0 %	172 000 ₽	73 725 ₽	315 000 ₽	9.20
Удалённая работа	1 762	1 282	427	24.2 %	250 000 ₽	200 000 ₽	325 000 ₽	8.76

8.1. Навыки и пары/тройки по регионам

Регион	Топ-3 навыка	Топ-2 пары	Топ-2 тройки	Повышенный спрос (индекс)
Москва	linux (60.0 %); kubernetes (55.1 %); docker (49.7 %)	docker;kubernetes (34.7 %); kubernetes;linux (33.2 %)	docker;kubernetes;linux (21.6 %); devops;docker;kubernetes (16.4 %)	yandex.cloud ×1.75; istio ×1.75
Санкт-Петербург	linux (61.3 %); docker (56.8 %); kubernetes (51.8 %)	docker;kubernetes (35.9 %); docker;linux (35.7 %)	docker;kubernetes;linux (22.5 %); ansible;docker;linux (18.9 %)	victoriametrics ×4.02; windows server ×2.84
Новосибирск	linux (69.9 %); docker (60.2 %); kubernetes (57.7 %)	docker;kubernetes (44.7 %); docker;linux (43.9 %)	docker;kubernetes;linux (35.8 %); ansible;docker;kubernetes (23.6 %)	tcp/ip ×2.46; aws ×1.96

Индекс спроса — доля навыка внутри региона, делённая на его долю во всей DevOps-выборке; индекс > 1 означает, что в регионе навык встречается чаще среднего. Учитываются навыки с частотой ≥ 10.

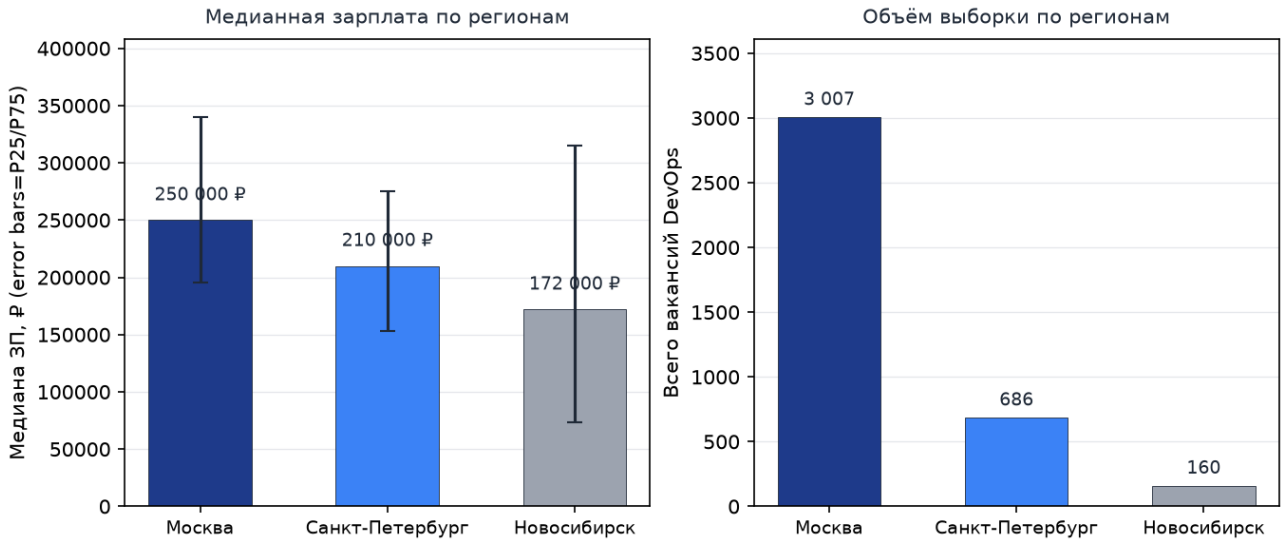


Рис. 12. Медианная зарплата (P25/P75) и объём выборки по регионам.

Наблюдения:

- Москва — 3 007 из 5 158 вакансий (58.3 %) и медиана 250 000 ₽; Санкт-Петербург — 686 (13.3 %) и 210 000 ₽; Новосибирск — 160 (3.1 %) и 172 000 ₽. Разрыв Москва/Новосибирск — 78 000 ₽ (45 % от новосибирской медианы).
- В Новосибирске необычно широкий интервал: P25 = 73 725 ₽, P75 = 315 000 ₽. Это не ошибка расчёта, а неоднородность выборки: город представлен и low-cost сопровождением, и высокооплачиваемыми удалёнными/продуктовыми командами. В Москве интервал уже (195 750–340 000 ₽).
- Среднее число навыков растёт от Москвы (7.65) к Санкт-Петербургу (8.34) и Новосибирску (9.20): в небольших городах вакансия чаще «широкая», требует больше технологий на одного инженера.
- Ядро стека одинаково во всех трёх регионах: linux, docker, kubernetes, ansible. Различаются акценты: Москва сильнее по yandex.cloud, istio, sonarqube (облако и платформа); Санкт-Петербург — по victoriametrics (×4.02), windows server и powershell; Новосибирск — по tcp/ip (×2.46), aws и java.
- Удалённая работа (1 762 вакансии, 34.2 % выборки) даёт медиану 250 000 ₽ — на уровне Москвы и выше Санкт-Петербурга; топ навык — kubernetes. Это показывает, что удалёнка не является «дешёвой» альтернативой офису.

9. Опыт и разведочный анализ

Опыт	Вакансий	С навыками	Медиана	P25	P75	Ср. навыков
Нет опыта	71	35	69 600 ₽	43 500 ₽	177 000 ₽	7.77
От 1 года до 3 лет	402	270	152 250 ₽	104 400 ₽	200 075 ₽	9.39
От 3 до 6 лет	566	426	250 000 ₽	200 000 ₽	326 250 ₽	8.87
Более 6 лет	71	56	304 500 ₽	275 000 ₽	402 500 ₽	8.27

Медиана монотонно растёт по грейдам: 69 600 → 152 250 → 250 000 → 304 500 ₽. При этом число навыков почти не меняется (7.8–9.4), то есть прибавка связана с уровнем ответственности, а не с формальным расширением стека. Крайние группы малочисленны (по 71 вакансии), поэтому P25/P75 у них широкие.

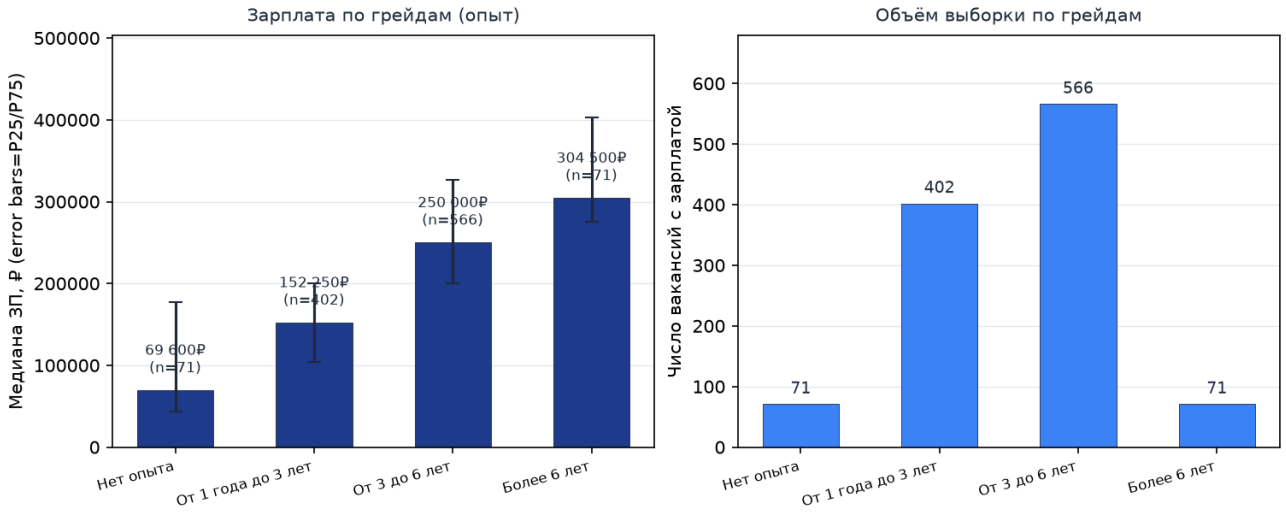


Рис. 13. Медианная зарплата и объём выборки по грейдам.

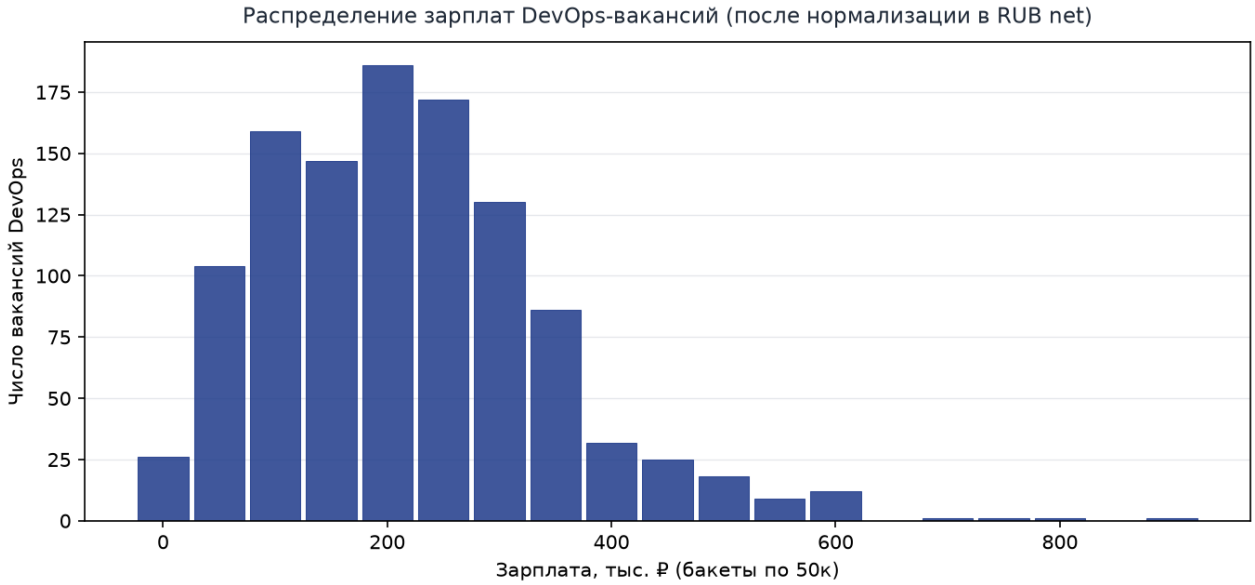


Рис. 14. Распределение зарплат DevOps-вакансий после нормализации в RUB net.

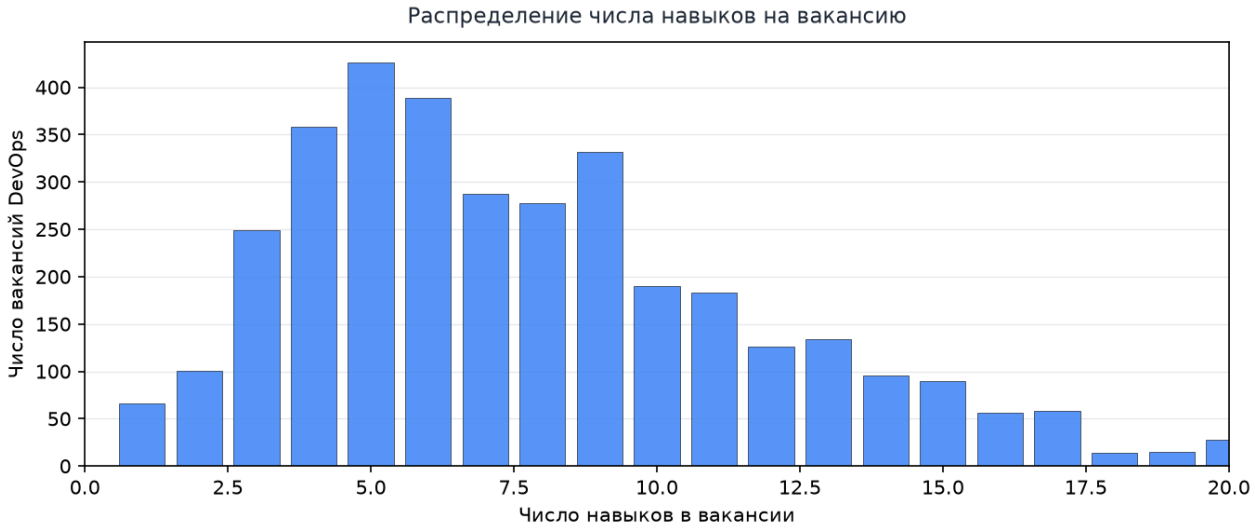


Рис. 15. Распределение числа навыков на вакансию (медиана 5, среднее 8.02).

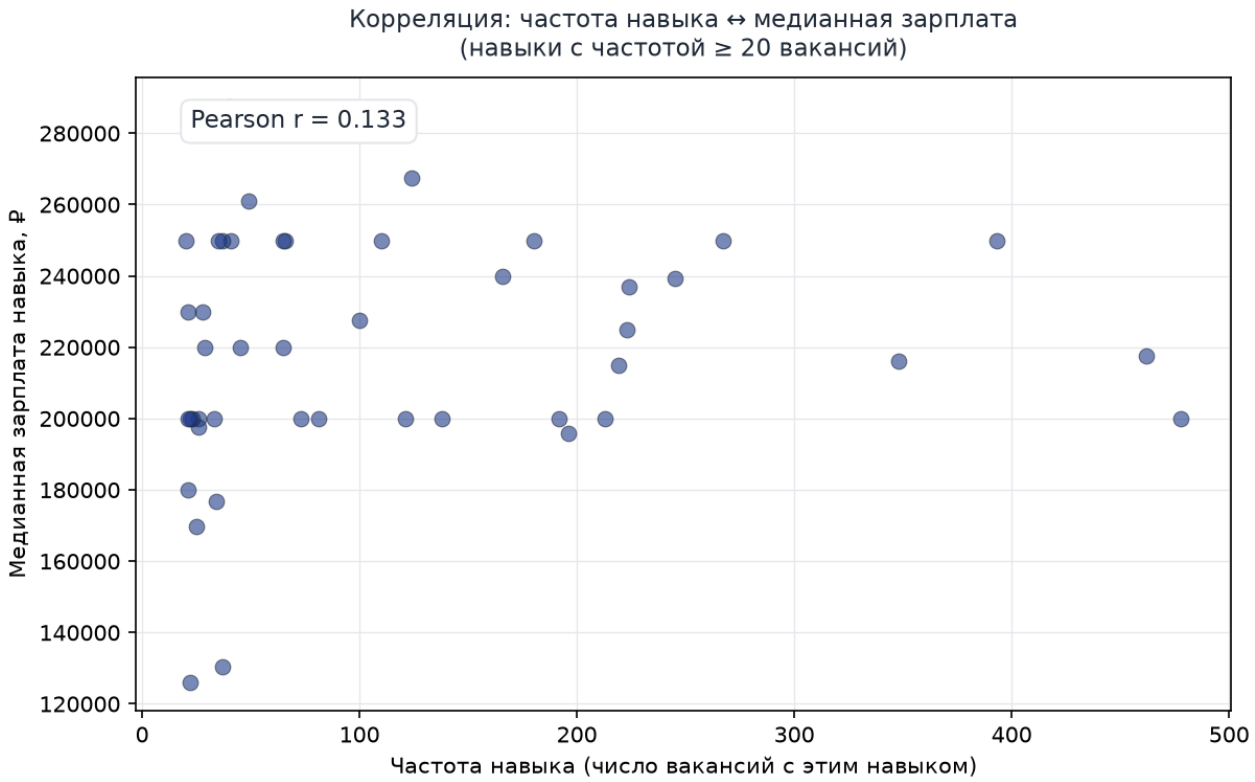


Рис. 16. Связь частоты навыка и его медианной зарплаты (навыки с частотой ≥ 20).

9.1. EDA: что ещё есть в датасете

Показатель	Значение
Всего валидных строк	229 387
Уникальных категорий role	17
Уникальных городов (area)	1 643
Уникальных работодателей (id)	31 796
DevOps-вакансий	5 158
Уникальных навыков в DevOps	1 401
Медиана навыков на вакансию	5
P10 / P90 зарплаты	90 000 ₽ / 375 000 ₽

Топ-роли (весь датасет)	N
Программист, разработчик	87 802
Аналитик	30 112
Системный администратор	29 723
Тестировщик	14 151
Специалист по информационной безопасности	13 453

Топ-города (весь датасет)	N
Москва	92 728
Санкт-Петербург	25 436
Екатеринбург	8 993
Новосибирск	6 909
Казань	5 598

Топ-10 навыков DevOps	N
linux	2 190
kubernetes	1 862
docker	1 850
devops	1 319
ansible	1 252

Топ-10 навыков DevOps	N
python	1 058
ci/cd	1 044
bash	925
postgresql	892
grafana	830

Поле employer_name в датасете пустое, поэтому топ работодателей считается по employer_id (например, 4649269 — 168 DevOps-вакансий). Всего 31 796 различных employer_id. Поле role содержит только 17 категорий, что подтверждает: это профессия, а не заголовок вакансии; заголовки лежат в name.

10. Выводы

- Инфраструктура и данные: RustFS поднят, бакет hh-lake содержит raw/vacancies.csv размером 847 619 384 B; размер и ETag подтверждены head-object, SHA-256 зафиксирован в отчёте.
- Все три реализации (наивный MapReduce, MapReduce по чанкам, DuckDB SQL) дали одинаковые 1 401 уникальный навык, 20 454 пары и 144 024 тройки; суммы частот 28 336 / 134 651 / 506 549 совпали до строки. Это подтверждает и корректность фильтра, и отсутствие потерь/дублей на границах чанков.
- Рост числа комбинаций объясняется C(n,k) внутри вакансии и тяжёлым хвостом длинных списков навыков: в среднем 8.02 навыка, поэтому эмиссий троек в 3.76 раза больше пар, а уникальных троек — в 7.0 раз больше уникальных пар.
- По времени DuckDB быстрее (8.52 с), но потоковый MapReduce v2 в 10.7 раза экономнее по памяти (132 MiB против 1 419 MiB). Для 808 MiB на машине с 15 GiB RAM оба варианта пригодны, но на больших объёмах преимущество потоковой схемы станет решающим.
- Оптимизация через curated-витрину дала 6.17× по времени и 1.17× по памяти при побитово том же результате: главный эффект — не читать 808 MiB и 23 колонки на каждом запуске.
- Зарплата: современный платформенный стек (SRE, OpenShift, Helm, Terraform, GitLab CI) повышает медиану на 45–100 %, а legacy-администрирование (Debian, docker-compose, «администрирование») понижает на 40–55 %. Это согласуется с гипотезой о переходе рынка от ручного сопровождения к автоматизации и IaC.
- Регионы: Москва платит медианно на 19 % больше Санкт-Петербурга и на 45 % больше Новосибирска; при этом ядро стека одинаково, а различия — в облачных и платформенных акцентах. Удалённая работа по медиане не уступает Москве.
- Ограничения: employer_name пуст; key_skills содержит не только технологии, но и теги роли; топовые пары/тройки опираются на 8–38 вакансий; все замеры локальные (RustFS на loopback), поэтому абсолютные времена не переносятся на облако.

Приложение А. Выводы команд

Ниже приведены фактические логи запусков (report/logs/). Команды запускались из корня репозитория в .venv; для профилирования и бенчмарка дополнительно сохраняются отдельные логи каждого прогона.

A.1. Проверка окружения

```
================================================================================================================
ENVIRONMENT CHECK — HW3 DevOps pipeline
================================================================================================================
  [OK  ] Python >= 3.10                          3.14.5 (C:\Users\Юлия\Documents\ITMO-Data-Engineering\.venv\Scr
ipts\python.exe)
  [OK  ] package boto3                           1.43.116
  [OK  ] package duckdb                          2.0.0
  [OK  ] package psutil                          7.2.2
  [OK  ] package matplotlib                      3.11.2
  [OK  ] package reportlab                       5.0.1
  [OK  ] package pypdf                           6.21.1
  [OK  ] S3 endpoint http://localhost:9010       TCP localhost:9010 доступен
  [OK  ] S3 client                               backend=boto3, bucket=hh-lake
  [OK  ] object s3://hh-lake/raw/vacancies.csv   847,619,384 B (808.4 MiB), etag=efb65458995932747001aa004592d19
c-102
  [OK  ] object curated/devops_skills.csv        322,653 B (0.31 MiB)
  [OK  ] dir report/logs                         есть
  [OK  ] dir report/images                       есть
  [OK  ] dir report/screenshots                  есть
  [OK  ] dir data                                есть
  [OK  ] docker compose                          hh-rustfs-home Up 31 minutes (healthy) | hh-s3cli Up 30 minutes
----------------------------------------------------------------------------------------------------------------
  OK: 16  WARN: 0  FAIL: 0

  Окружение готово к запуску: python src/run_all.py
```

A.2. MapReduce v1


```
{
  "file_size_bytes": 847619384,
  "filter_mode": "role",
  "n_chunks": 1,
  "rows_total": 229389,
  "rows_parsed": 229387,
  "rows_matched": 5158,
  "rows_with_skills": 3534,
  "reader_stats": {
    "lines_total": 229389,
    "rows_parsed": 229387,
    "skipped_blank": 1,
    "skipped_malformed": 1,
    "skipped_header": 0
  },
  "emissions_total": 669536,
  "n_skills_1": 1401,
  "n_skills_2": 20454,
  "n_skills_3": 144024,
  "total_frequency": {
    "1": 28336,
    "2": 134651,
    "3": 506549
  },
  "phase_time_read_map_s": 11.745,
  "phase_time_reduce_s": 0.356,
  "phase_time_write_s": 0.741,
  "phase_time_total_s": 12.842,
  "results": {
    "1": {
      "n_combinations": 1401,
      "total_frequency": 28336,
      "top": [
        [
          "linux"
        ],
        2190
      ],
      "s3_key": "results/mapreduce_v1/skills_1.csv",
      "local_path": "data/results_v1\\skills_1.csv"
    },
    "2": {
      "n_combinations": 20454,
      "total_frequency": 134651,
      "top": [
        [

          "docker",
          "kubernetes"
        ],
        1251
      ],
      "s3_key": "results/mapreduce_v1/skills_2.csv",
      "local_path": "data/results_v1\\skills_2.csv"
    },
    "3": {
      "n_combinations": 144024,
      "total_frequency": 506549,
      "top": [
        [
          "docker",
          "kubernetes",
          "linux"
        ],
        804
      ],
      "s3_key": "results/mapreduce_v1/skills_3.csv",
      "local_path": "data/results_v1\\skills_3.csv"
    }
  },
  "manifest": "data/manifests\\mapreduce_v1.json"
}
```

A.3. MapReduce v2


```
{
  "file_size_bytes": 847619384,
  "filter_mode": "role",
  "chunk_size_bytes": 8388608,
  "n_chunks": 102,
  "lines_total": 229389,
  "lines_prefiltered": 5784,
  "rows_matched": 5158,
  "rows_with_skills": 3534,
  "lines_malformed": 0,
  "events_per_k": {
    "1": 28336,
    "2": 134651,
    "3": 506549
  },
  "n_skills_1": 1401,
  "n_skills_2": 20454,
  "n_skills_3": 144024,
  "total_frequency": {
    "1": 28336,
    "2": 134651,
    "3": 506549
  },
  "phase_time_stat_s": 0.007,
  "phase_time_map_s": 11.241,
  "phase_time_reduce_write_s": 0.79,
  "phase_time_total_s": 12.038,
  "results": {
    "1": {
      "n_combinations": 1401,
      "total_frequency": 28336,
      "top": [
        [
          "linux"
        ],
        2190
      ],
      "s3_key": "results/mapreduce_v2/skills_1.csv",
      "local_path": "data/results_v2\\skills_1.csv"
    },
    "2": {
      "n_combinations": 20454,
      "total_frequency": 134651,
      "top": [
        [
          "docker",

          "kubernetes"
        ],
        1251
      ],
      "s3_key": "results/mapreduce_v2/skills_2.csv",
      "local_path": "data/results_v2\\skills_2.csv"
    },
    "3": {
      "n_combinations": 144024,
      "total_frequency": 506549,
      "top": [
        [
          "docker",
          "kubernetes",
          "linux"
        ],
        804
      ],
      "s3_key": "results/mapreduce_v2/skills_3.csv",
      "local_path": "data/results_v2\\skills_3.csv"
    }
  },
  "manifest": "data/manifests\\mapreduce_v2.json"
}
```

A.4. DuckDB SQL


```
{
  "csv_path": "s3://hh-lake/raw/vacancies.csv",
  "filter_mode": "role",
  "rows_matched": 5158,
  "rows_with_skills": 3534,
  "n_skills_1": 1401,
  "n_skills_2": 20454,
  "n_skills_3": 144024,
  "total_frequency": {
    "1": 28336,
    "2": 134651,
    "3": 506549
  },
  "duckdb_version": "2.0.0",
  "phase_time_query_s": 8.927,
  "phase_time_write_s": 1.3,
  "phase_time_total_s": 10.227,
  "results": {
    "1": {
      "n_combinations": 1401,
      "total_frequency": 28336,
      "top": [
        [
          "linux"
        ],
        2190
      ],
      "s3_key": "results/duckdb/skills_1.csv",
      "local_path": "data/results_duckdb\\skills_1.csv"
    },
    "2": {
      "n_combinations": 20454,
      "total_frequency": 134651,
      "top": [
        [
          "docker",
          "kubernetes"
        ],
        1251
      ],
      "s3_key": "results/duckdb/skills_2.csv",
      "local_path": "data/results_duckdb\\skills_2.csv"
    },
    "3": {
      "n_combinations": 144024,
      "total_frequency": 506549,

      "top": [
        [
          "docker",
          "kubernetes",
          "linux"
        ],
        804
      ],
      "s3_key": "results/duckdb/skills_3.csv",
      "local_path": "data/results_duckdb\\skills_3.csv"
    }
  },
  "manifest": "data/manifests\\duckdb.json"
}
```

A.5. Верификация

```
=== Verification summary ===
all_match: True

stack  left     right       rows_a    rows_b     sum_a     sum_b  match
-------------------------------------------------------------------------
1      v1       v2            1401      1401     28336     28336  True
1      v1       duckdb        1401      1401     28336     28336  True
1      v2       duckdb        1401      1401     28336     28336  True
2      v1       v2           20454     20454    134651    134651  True
2      v1       duckdb       20454     20454    134651    134651  True
2      v2       duckdb       20454     20454    134651    134651  True
3      v1       v2          144024    144024    506549    506549  True
3      v1       duckdb      144024    144024    506549    506549  True
3      v2       duckdb      144024    144024    506549    506549  True

=== Manifest checks ===
filter modes: ['role']
rows matched: {'v1': 5158, 'v2': 5158, 'duckdb': 5158}
  ✓ manifests consistent

✓ ВСЕ РЕЗУЛЬТАТЫ СОВПАЛИ (v1 == v2 == duckdb)
```

A.6. Зарплатный анализ

```
overall_median: 210000 ₽ (787 vacancies with skills+salary)
all salary rows (incl. no skills): 1110, median 215000 ₽
n_pairs_total: 582, n_triples_total: 2880
audit stats: {"rows_matched": 5158, "rows_with_skills": 3534, "rows_with_convertible_salary": 1110, "rows_with_s
kills_and_salary": 787, "currencies_seen": {"RUR": 1037, "USD": 53, "EUR": 20, "BYR": 1}, "filter_mode": "role",
 "reader": {"lines_total": 229389, "rows_parsed": 229387, "skipped_blank": 1, "skipped_malformed": 1, "skipped_h
eader": 0}}

=== TOP-10 PAIRS RAISE ===
   +133925 ₽ ( +61.9%)  devops;sre                                    med=350175 (n=12)  base devops=216250 (n=
348)
   +112925 ₽ ( +47.6%)  grafana;openshift                             med=350000 (n=12)  base grafana=237075 (n
=224)
   +112250 ₽ ( +52.2%)  bash;мониторинг                               med=327250 (n=10)  base bash=215000 (n=21
9)
   +105000 ₽ ( +48.3%)  docker;openshift                              med=322500 (n=12)  base docker=217500 (n=
462)
   +104500 ₽ ( +52.2%)  linux;openshift                               med=304500 (n=13)  base linux=200000 (n=4
78)
   +104250 ₽ ( +53.3%)  git;gitlab ci                                 med=300000 (n=19)  base git=195750 (n=196
)
   +101375 ₽ ( +47.2%)  bash;субд                                     med=316375 (n=10)  base bash=215000 (n=21
9)
   +100625 ₽ ( +44.7%)  ci/cd;helm                                    med=325625 (n=12)  base ci/cd=225000 (n=2
23)
   +100000 ₽ ( +50.0%)  terraform;zabbix                              med=300000 (n=38)  base zabbix=200000 (n=
138)
   +100000 ₽ ( +50.0%)  kafka;zabbix                                  med=300000 (n=13)  base zabbix=200000 (n=
138)

=== TOP-10 PAIRS LOWER ===
   -137500 ₽ ( -55.0%)  ansible;администрирование                      med=112500 (n=11)  base ansible=250000 (n
=267)
   -125000 ₽ ( -50.0%)  atlassian jira;prometheus                     med=125000 (n=10)  base prometheus=250000
 (n=180)
   -110000 ₽ ( -44.0%)  ansible;debian                                med=140000 (n=11)  base ansible=250000 (n
=267)
   -108750 ₽ ( -45.5%)  debian;postgresql                             med=130500 (n=11)  base postgresql=239250
 (n=245)
   -100000 ₽ ( -50.0%)  debian;linux                                  med=100000 (n=19)  base linux=200000 (n=4
78)
   -100000 ₽ ( -50.0%)  debian;nginx                                  med=100000 (n=12)  base nginx=200000 (n=1
92)
   -100000 ₽ ( -50.0%)  apache http server;администрирование серверов linux  med=100000 (n=11)  base администрир
ование серверов linux=200000 (n=121)
   -100000 ₽ ( -50.0%)  debian;администрирование серверов linux       med=100000 (n=10)  base администрирование
```

```
 серверов linux=200000 (n=121)
    -97075 □ ( -40.9%)  docker-compose;grafana                          med=140000 (n=10)  base grafana=237075 (n
=224)
    -95000 □ ( -38.0%)  ansible;atlassian jira                          med=155000 (n=12)  base ansible=250000 (n
=267)

=== TOP-10 TRIPLES RAISE ===
   +180000 □ (+100.0%)  git;gitlab ci;python                            med=360000 (n=9)  base git;python=180000
(n=76)
   +160000 □ ( +80.0%)  git;gitlab ci;postgresql                        med=360000 (n=11)  base git;postgresql=20
0000 (n=84)
   +151200 □ ( +72.4%)  gitlab ci;linux;python                          med=360000 (n=9)  base linux;python=20880
0 (n=151)
   +150625 □ ( +60.2%)  grafana;helm;terraform                          med=400625 (n=8)  base grafana;terraform=
250000 (n=72)
   +150000 □ ( +75.0%)  ci/cd;python;zabbix                             med=350000 (n=17)  base ci/cd;python=2000
00 (n=69)
   +150000 □ ( +75.0%)  ci/cd;elk;python                                med=350000 (n=15)  base ci/cd;python=2000
00 (n=69)
   +150000 □ ( +75.0%)  ci/cd;git;jenkins                               med=350000 (n=13)  base ci/cd;git=200000
(n=75)
   +150000 □ ( +75.0%)  git;gitlab;jenkins                              med=350000 (n=11)  base git;gitlab=200000
 (n=46)
   +150000 □ ( +75.0%)  bash;python;субд                                med=350000 (n=8)  base bash;python=200000
 (n=124)
   +150000 □ ( +75.0%)  ci/cd;python;удаленная работа                   med=350000 (n=8)  base ci/cd;python=20000
0 (n=69)

=== TOP-10 TRIPLES LOWER ===
   -145000 □ ( -59.2%)  docker-compose;linux;prometheus                 med=100000 (n=9)  base linux;prometheus=2
45000 (n=132)
   -132125 □ ( -56.9%)  jenkins;postgresql;sql                          med=100000 (n=9)  base jenkins;postgresql
=232125 (n=38)
   -125000 □ ( -50.0%)  elk;prometheus;администрирование серверов linux  med=125000 (n=19)  base elk;prometheus=
250000 (n=64)
   -125000 □ ( -50.0%)  ansible;atlassian jira;linux                    med=125000 (n=10)  base ansible;linux=250
000 (n=190)
   -125000 □ ( -50.0%)  ansible;atlassian jira;prometheus               med=125000 (n=10)  base ansible;prometheu
s=250000 (n=114)
   -125000 □ ( -50.0%)  kubernetes;postgresql;субд                      med=125000 (n=9)  base kubernetes;postgre
sql=250000 (n=137)
   -125000 □ ( -50.0%)  postgresql;prometheus;английский язык           med=125000 (n=9)  base postgresql;prometh
eus=250000 (n=74)
   -125000 □ ( -50.0%)  atlassian jira;devops;prometheus                med=125000 (n=8)  base devops;prometheus=
250000 (n=100)
   -122250 □ ( -48.9%)  ansible;python;английский язык                  med=127750 (n=10)  base ansible;python=25

0000 (n=90)
   -120000 □ ( -49.0%)  atlassian jira;linux;prometheus                 med=125000 (n=10)  base linux;prometheus=
245000 (n=132)
```

A.7. Регионы

```
DevOps вакансий всего: 5158 (с навыками: 3534)

=== Топ-10 регионов (поле area) ===
  Москва                          3007 ✓
  Санкт-Петербург                  686 ✓
  Екатеринбург                     188
  Новосибирск                      160 ✓
  Казань                           130
  Нижний Новгород                  116
  Ростов-на-Дону                    74
  Самара                            71
  Томск                             51
  Краснодар                         48

=== Топ-5 типов занятости (поле schedule) ===
  Полный день                     3225
  Удаленная работа                1762
  Гибкий график                    157
  Сменный график                    12
  Вахтовый метод                     2

Анализируемые регионы (REGIONS): ['Москва', 'Санкт-Петербург', 'Новосибирск']
  (изменить: $env:REGIONS='Москва,Санкт-Петербург,Екатеринбург')

=== Сводка по регионам ===
Регион                        N  N нав   N ЗП   Медиана       P25       P75  #нав
-------------------------------------------------------------------------------------
Москва                     3007   2015    573   250000₽   195750₽   340000₽  7.65
Санкт-Петербург             686    488    161   210000₽   153000₽   275000₽  8.34
Новосибирск                 160    123     48   172000₽    73725₽   315000₽   9.2

=== Удалённая работа (отдельный срез) ===
  n=1762  skills=1282  salary=427  median=250000₽  top=kubernetes

=== Топ-5 навыков по регионам ===

[Москва]  median=250000₽  n=3007  mean_skills=7.65
  linux                    1210 вакансий  (60.0%)
  kubernetes               1111 вакансий  (55.1%)
  docker                   1001 вакансий  (49.7%)
  devops                    768 вакансий  (38.1%)
  ansible                   691 вакансий  (34.3%)

[Санкт-Петербург]  median=210000₽  n=686  mean_skills=8.34
  linux                     299 вакансий  (61.3%)
  docker                    277 вакансий  (56.8%)

  kubernetes                253 вакансий  (51.8%)
  ansible                   199 вакансий  (40.8%)
  ci/cd                     167 вакансий  (34.2%)

[Новосибирск]  median=172000₽  n=160  mean_skills=9.2
  linux                      86 вакансий  (69.9%)
  docker                     74 вакансий  (60.2%)
  kubernetes                 71 вакансий  (57.7%)
  ansible                    59 вакансий  (48.0%)
  python                     49 вакансий  (39.8%)
```

A.8. Опыт

```
DevOps вакансий с зарплатой: 1110
{"rows_matched": 5158, "rows_with_convertible_salary": 1110, "filter_mode": "role", "reader": {"lines_total": 22
9389, "rows_parsed": 229387, "skipped_blank": 1, "skipped_malformed": 1, "skipped_header": 0}}

=== Зарплата по грейдам (опыт) ===
Опыт                            N     Медиана         P25         P75  #нав
-------------------------------------------------------------------------------
Нет опыта                      71      69600₽      43500₽     177000₽  7.77
От 1 года до 3 лет            402     152250₽     104400₽     200075₽  9.39
От 3 до 6 лет                 566     250000₽     200000₽     326250₽  8.87
Более 6 лет                    71     304500₽     275000₽     402500₽  8.27

=== Топ-5 навыков по грейдам ===

[Нет опыта]
  linux                          23
  docker                         15
  kubernetes                     14
  devops                          8
  nginx                           7

[От 1 года до 3 лет]
  linux                         172
  docker                        163
  kubernetes                    116
  devops                        113
  python                         94

[От 3 до 6 лет]
  docker                        256
  linux                         252
  kubernetes                    231
  devops                        205
  ansible                       157

[Более 6 лет]
  kubernetes                     32
  linux                          31
  docker                         28
  devops                         22
  ansible                        20
```

A.9. Профилирование (итог)

```
=== Profile Summary ===
Method                              n_ok   wall_med_s   wall_min_s   wall_max_s   peak_med_MiB   peak_max_MiB
-------------------------------------------------------------------------------------------------------------
MapReduce v1 (naive)                 3/3       13.310       12.698       13.373         196.33         204.02
MapReduce v2 (optimized)             3/3       12.985       12.552       13.692         132.40         132.97
DuckDB SQL                           3/3        8.515        7.549        9.317        1418.50        1723.26

Environment: {'python': '3.14.5', 'platform': 'win32', 'cpu_count': 16, 'total_memory_gib': 15.34, 'available_me
mory_gib_at_end': 1.6}
```

A.10. Бенчмарк оптимизации

```
=== Optimization Benchmark Summary ===
Method                                  n_ok   wall_med_s   wall_min_s   wall_max_s   peak_med_MiB
-----------------------------------------------------------------------------------------------------
MapReduce v2 (raw CSV)                   3/3       13.162       12.916       13.314         132.48
MapReduce v2 (curated CSV)               3/3        2.133        2.055        2.175         113.35

Outputs identical (raw vs curated): True
Speedup:  6.17× faster (curated vs raw)
Memory:   1.17× less (curated vs raw)
```

A.11. Графики

```
Charts generated in report\images: 9/9
  01_growth.png  (67 KiB)
  02_profile.png  (57 KiB)
  03_salary_pairs.png  (68 KiB)
  04_regions.png  (60 KiB)
  05_optimization.png  (53 KiB)
  06_salary_histogram.png  (36 KiB)
  07_skills_per_vacancy.png  (34 KiB)
  08_freq_vs_salary.png  (58 KiB)
  09_experience.png  (68 KiB)
```

Приложение B. Воспроизводимость

Полный пайплайн запускается одной командой (идемпотентно, с остановкой на первой ошибке):

```
docker compose up -d
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python -m unittest discover -s tests -v
.venv\Scripts\python src\check_env.py
.venv\Scripts\python src\run_all.py
# только отчёт по уже готовым данным:
.venv\Scripts\python src\make_report.py
```

Переменные окружения: S3_ENDPOINT (по умолчанию http://localhost:9010), S3_ACCESS_KEY / S3_SECRET_KEY (hhadmin / hhadmin-secret), S3_BUCKET (hh-lake), VACANCIES_KEY (raw/vacancies.csv), VARIANT_FILTER_MODE (role), REGIONS (Москва,Санкт-Петербург,Новосибирск), PROFILE_RUNS (3). Тесты используют локальный backend и не требуют Docker.

Приложение C. Ограничения и честность измерений

- Числа в отчёте получены 26.10.2026 на одной машине; при повторном запуске времена могут отличаться на 5–15 %, но контрольные числа (5 158 / 1 401 / 20 454 / 144 024 и суммы частот) воспроизводимы детерминированно.
- Пиковая память — RSS дерева процессов, а не точный объём Python-кучи; для DuckDB в RSS входят буферы httpfs и кэш страниц, поэтому цифра выше, чем объём самих результатов.
- Проценты зарплатных прибавок — медианы по малым группам; они показывают направление и порядок, а не гарантированный эффект навыка. База для пары — более частый навык, для тройки — более частая пара.
- Фильтр role == "DevOps-инженер" выбран как основной; широкий режим role_or_name даёт другие числа (7 786 / 1 918 / 31 060 / 225 224), которые также сверены между v1, v2 и DuckDB.
- Скриншоты хранилища — реальные снимки (docker compose ps, aws-cli, веб-консоль RustFS, вывод verify/profile/benchmark), а не рисунки; их можно перегенерировать командой python src/make_screenshots.py.